Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 7 — Solutions

Tuples, sets and dictionaries

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — Tuples and sets

A1 *Eight lines with tuples*

	line	value
1	`t[1]`	`7`
2	`t[-1] + t[0]`	`5`, which is `1 + 4`
3	`t + (2,)`	`(4, 7, 1, 2)`, a new tuple
4	`len((5,))`	`1`
5	`type((5))`	`int`, the brackets only group
6	`a, b = b, a + b`	`a` is `2` and `b` is `3`
7	`a, b, c = (1, 2)`	`ValueError`, not enough values to unpack
8	`t[0] = 9`	`TypeError`, a tuple cannot be changed

Number 6. The right-hand side is worked out completely first, with the old values. That gives the tuple `(2, 1 + 2)`, which is `(2, 3)`. Only then is it unpacked into `a` and `b`. Anyone who got `b = 4` updated `a` first and then used the new value. That is what the two lines would do if written separately, and exactly what the one line avoids.

Worth noticing. Number 3 does not contradict number 8. `t + (2,)` builds a *new* tuple and leaves `t` alone, just as `"cat" + "s"` did with strings in lecture 5. Immutable means you cannot change the tuple, not that you cannot make new ones from it.

A2 *Eight lines with sets*

	line	value
1	`len(s)`	`4`
2	`1 in s`	`True`
3	`s | {9}`	`{1, 3, 4, 5, 9}`
4	`s & {1, 2, 3}`	`{1, 3}`
5	`s - {1, 5}`	`{3, 4}`
6	`len(set("mississippi"))`	`4`, the letters m, i, s, p
7	`type({})`	`dict`
8	`s[0]`	`TypeError`, a set has no positions

Number 1 is 4 and not 5, because the second `1` is the same value and a set holds each value once. **Number 7** is a dictionary because `{}` was taken by dictionaries first. The empty set has to be written `set()`.

A3 *Which structure?*

a) **List.** The question is "what was move number 12?", and the order is the whole point.

b) **Tuple.** A colour is one thing in three parts, and it always has exactly three.

c) **Set.** The only question is "is Japan in there?", and each country appears once.

d) **Dictionary.** The question is "what is Ada's number?", a lookup by name.

e) **List.** The question is "how long is chapter 3?", by position, in order.

f) **Dictionary of tuples.** The outer question is "when is it open on Monday?", a lookup by day. Each answer is one thing in two parts, opening and closing time: `{"Monday": (9, 18), "Sunday": (10, 13), ...}`.

Worth noticing. Other answers can be defended. A list of seven tuples for number 6 works, if you remember that position 0 is Monday. The dictionary is better because the code then says `hours["Monday"]` instead of `hours[0]`. What matters is that the justification starts from the question, not from the data.

A4 *The letters a sentence never uses*

```
# read the sentence
# missing letters: the alphabet minus the sentence's letters
# print them in order, or say that every letter is used

sentence = input("A sentence: ")

alphabet = set("abcdefghijklmnopqrstuvwxyz")
missing = alphabet - set(sentence.lower())

if len(missing) == 0:
    print("It uses every letter.")
else:
    print(sorted(missing))
```

The fox sentence uses every letter. `Hello, world!` misses nineteen of them, starting with `a`, `b`, `c`, `f`.

Spaces and punctuation do not need removing. They end up in `set(sentence)`, but the subtraction starts from the alphabet. Taking away a comma that was never in the alphabet changes nothing.

Worth noticing. Lecture 5 would have needed a loop over the alphabet with `in` for each letter. With sets the whole question is one subtraction, and it reads like the question itself: the alphabet, minus what the sentence uses.

A5 *A list of points*

```
points = [(3, 4), (-1, 2), (0, -5), (6, 1), (-2, -2)]

# a) print each point
for x, y in points:
    print(f"x = {x}, y = {y}")

# b) above the horizontal axis
above = 0
for x, y in points:
    if y > 0:
        above += 1
print(above)                    # 3

# c) closest to (0, 0), comparing x*x + y*y
closest = points[0]
best = closest[0] ** 2 + closest[1] ** 2
for x, y in points:
    d2 = x * x + y * y
    if d2 < best:
        closest, best = (x, y), d2
print(closest)                  # (-1, 2)
```

No square root needed. The square root keeps the order of positive numbers: if $a < b$, then $\sqrt{a} < \sqrt{b}$. So the point with the smallest $x^2 + y^2$ also has the smallest distance. Skipping the square root saves work and avoids floats altogether.

d) `[(-2, -2), (-1, 2), (0, -5), (3, 4), (6, 1)]`. Tuples are compared part by part, so the points are sorted by `x`, and `y` only decides between points with the same `x`.

Part B — Dictionaries

B1 *Ten lines with one dictionary*

After the three lines, `d` is `{"a": 5, "b": 2, "c": 3}`.

	line	value
1	`len(d)`	`3`
2	`d["a"]`	`5`, replaced, not added
3	`d.get("z")`	`None`
4	`d.get("z", 0)`	`0`
5	`"b" in d`	`True`
6	`2 in d`	`False`, `in` looks at the keys
7	`list(d)`	`["a", "b", "c"]`
8	`list(d.values())`	`[5, 2, 3]`
9	`d["z"]`	`KeyError: 'z'`
10	`{"x": 1, "x": 2}`	`{"x": 2}`, the last value wins

Number 8 keeps `"a"` in first place. Replacing a value does not move its key to the end.

B2 *It crashes on the first item*

a) `KeyError: 'red'`, on the very first pass.

b) `counts[colour] += 1` means `counts[colour] = counts[colour] + 1`. The right-hand side is worked out first, and it has to *read* `counts["red"]`. On the first pass there is no such key.

c) With `get`:

```
for colour in ["red", "blue", "red"]:
    counts[colour] = counts.get(colour, 0) + 1
```

d) With `in`:

```
for colour in ["red", "blue", "red"]:
    if colour in counts:
        counts[colour] += 1
    else:
        counts[colour] = 1
```

Both give `{"red": 2, "blue": 1}`. The second one says exactly what the first one does: the first time a colour appears is a special case, and `get(colour, 0)` handles it in the same line.

B3 *Who won the vote?*

```
counts = {}
for v in votes:
    counts[v] = counts.get(v, 0) + 1
print(counts)                   # {"pizza": 3, "sushi": 3, "pasta": 2}

top = max(counts.values())
winners = []
for option, n in counts.items():
    if n == top:
        winners.append(option)
print(winners)                  # ["pizza", "sushi"]
```

It is a tie. A running maximum with `if n > best` reports `pizza` only, because pizza appears first in the dictionary and sushi never beats it, it only equals it. With >= it reports `sushi` only. Both are wrong, and they are wrong silently.

The fix splits the question in two. First, what is the highest count? Then, which options have it?

Worth noticing. The data was chosen to tie. When you test a "find the best" program, a tie is the first case to try, and it is the one most people forget.

B4 *Word by word*

```
translated = []
for w in "the cat eats the fish quickly".split():
    translated.append(italian.get(w, w))
print(" ".join(translated))     # il gatto mangia il pesce quickly
```

`get(w, w)` reads as "the translation of `w`, or `w` itself if there is none". The default can be any value, including the key.

Real Italian fails word by word because words depend on each other. *Il pesce* is right, but *lo studente* and *la gatta* need a different article, decided by the next word, and word order differs between languages.

B5 *Turn it round*

a)

```
by_capital = {}
for country, capital in capitals.items():
    by_capital[capital] = country
```

b) The same loop on `grades` gives `{30: "Grace", 27: "Linus", 24: "Tim"}`. Ada and Alan are gone. Two students share each of the grades 30 and 27, and a key holds one value. `by_grade[30] = "Grace"` silently replaced `"Ada"`.

c) Make each value a list, and add to it.

```
by_grade = {}
for name, g in grades.items():
    if g not in by_grade:
        by_grade[g] = []
    by_grade[g].append(name)

# {30: ["Ada", "Grace"], 27: ["Alan", "Linus"], 24: ["Tim"]}
```

Worth noticing. Turning a dictionary round only works when the values are unique. Capitals are, grades are not. Grouping is the general version, and it is the same shape as counting: start the entry the first time you meet a key, then update it. Counting adds one, grouping appends.

B6 *Student records*

```
# 1.
print(records["Ada"]["Statistics"])          # 28

# 2. and 5.
for name, courses in records.items():
    if len(courses) == 0:
        print(f"{name:<6} no grades")
    else:
        mean = sum(courses.values()) / len(courses)
        print(f"{name:<6} {mean:.1f}")       # Ada 29.0 ...

# 3.
for name, courses in records.items():
    if "Programming" in courses:
        print(name)                          # Ada, Alan

# 4.
records["Alan"]["Ethics"] = 26
```

Part 5. Without the check, Tim's empty dictionary gives `sum([]) / len({})`, which is `0 / 0`, and the program stops with `ZeroDivisionError`. The same trap as the mean of no numbers in lecture 4.

Read `records["Ada"]["Statistics"]` left to right. `records["Ada"]` is Ada's dictionary of courses, and `["Statistics"]` looks up a key in *that*. After part 4, Alan's mean becomes `26.5`.

If you finish early

```
groups = {}
for w in words:
    key = tuple(sorted(w))
    if key not in groups:
        groups[key] = []
    groups[key].append(w)

for g in groups.values():
    print(g)

# ["listen", "silent", "enlist", "inlets"]
# ["google", "gogole"]
# ["banana"]
```

Both `sorted("listen")` and `sorted("silent")` give the same list.

```
["e", "i", "l", "n", "s", "t"]
```

A list cannot be a key, because it could change after it was stored. `tuple(...)` turns it into something that cannot, which is exactly what the slide on tuples promised. `"".join(sorted(w))` works too, since a

string is immutable as well.

This is grouping from B5 again, with a key you compute instead of one you are given.